MODUL 3 PROTOKOL KOMUNIKASI

Kaira Meilasya Nayada (H1H024020)
Asisten: Muhammad Nur Bijak B
Tanggal Percobaan: 15/04/2026
TK244005-Praktikum Sistem Mikrokontroller
Laboratorium Pemrograman – Jurusan Informatika UNSOED

Abstrak

Percobaan pada modul ini bertujuan untuk memahami dan mengimplementasikan protokol komunikasi serial pada Arduino Uno, meliputi UART dan I2C. Percobaan 3A berfokus pada pengendalian LED melalui perintah karakter dari Serial Monitor menggunakan UART. Percobaan 3B mengimplementasikan pembacaan nilai analog dari potensiometer dan menampilkan hasilnya pada LCD berbasis I2C. Metode yang digunakan mencakup konfigurasi pin sesuai tabel wiring, pemrograman dengan library Wire.h dan LiquidCrystal_I2C.h, serta penggunaan Serial.available() dan Serial.read() untuk komunikasi UART. Hasil menunjukkan bahwa LED berhasil dikendalikan via Serial Monitor, dan LCD I2C berhasil menampilkan nilai ADC beserta representasi bar yang proporsional sesuai putaran potensiometer.

Kata kunci: Arduino Uno, UART, I2C, LCD, Potensiometer, Protokol Komunikasi Serial.

1. PENDAHULUAN

Dalam sistem tertanam (embedded system), kemampuan berkomunikasi antara mikrokontroler dengan perangkat lain merupakan kebutuhan fundamental. Arduino Uno sebagai salah satu platform mikrokontroler yang paling banyak digunakan mendukung beberapa protokol komunikasi serial, di antaranya UART, I2C, dan SPI. Setiap protokol memiliki karakteristik, kelebihan, dan keterbatasan masing-masing yang menentukan kesesuaiannya dengan aplikasi tertentu [1].

Dilakukan dua percobaan utama. Percobaan 3A berfokus pada implementasi komunikasi UART antara Arduino dan komputer melalui Serial Monitor untuk mengendalikan LED. Percobaan 3B mengeksplorasi protokol I2C dengan menghubungkan LCD 16x2 dan potensiometer ke Arduino, sehingga nilai pembacaan ADC dapat ditampilkan secara visual pada layar LCD [2].

2. STUDI PUSTAKA

2.1 UNIVERSAL ASYNCHRONOUS RECEIVER-TRANSMITTER (UART)

UART adalah protokol komunikasi serial asinkron yang memungkinkan pertukaran data antara dua perangkat tanpa bergantung pada sinyal clock bersama. Setiap frame data dalam UART terdiri atas start bit, bit-bit data (biasanya 8 bit), bit paritas opsional, dan stop bit. Kecepatan transfer data ditentukan oleh baud rate yang harus dikonfigurasi sama di sisi pengirim maupun penerima agar komunikasi dapat berlangsung dengan benar [1].

Pada Arduino, komunikasi UART diimplementasikan melalui kelas *Serial* dengan metode utama *begin()*, *available()*, *read()*, dan *println()*. Baud rate yang umum digunakan adalah 9600 bps atau 115200 bps tergantung kebutuhan [3].

2.2 INTER-INTEGRATED CIRCUIT (I2C)

I2C adalah protokol komunikasi serial sinkron yang dirancang untuk menghubungkan beberapa perangkat (slave) ke satu atau lebih pengendali (master) menggunakan hanya dua jalur sinyal, yaitu SDA (Serial Data) dan SCL (Serial Clock). Setiap perangkat slave memiliki alamat unik 7-bit yang digunakan oleh master untuk mengidentifikasi perangkat tujuan [4].

Pada Arduino Uno, pin SDA terhubung ke pin A4 dan SCL ke pin A5. Protokol I2C mendukung kecepatan standar 100 kHz dan mode cepat 400 kHz. Keunggulan utama I2C adalah efisiensi kabel karena banyak perangkat dapat berbagi dua jalur yang sama. LCD 16×2 berbasis modul backpack PCF8574 memiliki alamat default 0x27 atau 0x3F [5].

2.3 SERIAL PERIPHERAL INTERFACE (SPI)

SPI adalah protokol komunikasi serial sinkron full-duplex yang dikembangkan oleh Motorola pada pertengahan 1980-an. SPI menggunakan empat jalur sinyal: SCLK (clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), dan SS (Slave Select). SPI umumnya digunakan untuk perangkat berkecepatan tinggi seperti memori flash, display TFT, dan kartu SD karena tidak memerlukan overhead pengalamatan seperti I2C [2].

3. METODOLOGI

3.1 ALAT DAN BAHAN

Berikut adalah komponen yang digunakan pada seluruh percobaan pada modul protokol komunikasi:

- Arduino Uno (berbasis ATmega328P)
- LED
- LCD 16x2 dengan modul I2C (backpack PCF8574)
- Potensiometer 10k Ohm
- Kabel Jumper
- Breadboard
- Komputer dengan Arduino IDE

3.2 PERCOBAAN 3A: KOMUNIKASI SERIAL UART

Rangkaian Rangkaian percobaan 3A terdiri dari satu buah LED yang dihubungkan ke pin digital 8 Arduino melalui resistor 220 Ohm, dengan kaki negatif LED terhubung ke GND. Program dikonfigurasi menggunakan komunikasi serial dengan baud rate 9600. Pada bagian setup(), pin LED dinyatakan sebagai OUTPUT dan Serial Monitor diinisialisasi. Pada bagian loop(), program memeriksa ketersediaan data serial menggunakan Serial.available(). Jika karakter '1' diterima maka LED dinyalakan, jika '0' diterima maka LED dimatikan, dan karakter lain menghasilkan pesan error pada Serial Monitor. Sketch disimpan dengan nama modul3_serialmonitor.

3.3 PERCOBAAN 3B: INTER-INTEGRATED CIRCUIT (I2C)

Rangkaian percobaan 3B menghubungkan LCD I2C dan potensiometer ke Arduino Uno. Pin SDA LCD I2C dihubungkan ke A4 dan SCL ke A5 Arduino. Kedua kaki luar potensiometer dihubungkan ke 5V dan GND, sementara kaki tengah (wiper) dihubungkan ke pin analog A0 sebagai input ADC. Program menggunakan library Wire.h dan LiquidCrystal_I2C.h dengan alamat LCD 0x27. Pada fungsi loop(), nilai analog dibaca menggunakan analogRead() menghasilkan nilai 0–1023, kemudian dipetakan ke skala 0–16 menggunakan map() untuk menentukan panjang bar pada baris kedua LCD. Baris pertama menampilkan nilai ADC dan baris kedua menampilkan representasi bar dengan delay 200 ms.

4. HASIL PERCOBAAN 3A: KOMUNIKASI SERIAL UART

Setelah program diupload, percobaan menunjukkan hasil yang sesuai dengan spesifikasi. Ketika karakter '1' dikirimkan melalui Serial Monitor, LED menyala dan Serial Monitor menampilkan pesan "LED ON". Ketika karakter '0' dikirimkan, LED padam dan Serial Monitor menampilkan "LED OFF". Karakter lain menghasilkan pesan "Perintah tidak dikenal", menandakan mekanisme validasi input berjalan dengan benar.

Penggunaan Serial.available() sangat krusial karena memeriksa apakah ada byte tersedia di buffer serial sebelum dilakukan pembacaan. Tanpa pengecekan ini, Serial.read() akan terus mengembalikan nilai -1 yang dapat menyebabkan pesan error terus muncul dan membebani sistem secara tidak perlu. Mekanisme ini juga menjaga sistem tetap responsif terhadap input baru [1].

Jawaban Pertanyaan Percobaan 3A:

1. Proses dari input keyboard hingga LED menyala/mati: pengguna mengetik karakter di Serial Monitor lalu klik Send, karakter dikirim via USB ke pin RX Arduino dan masuk ke buffer UART. Setiap iterasi loop() memeriksa buffer dengan Serial.available(). Jika ada data, Serial.read() mengambil karakter tersebut. Jika karakter adalah '1', program memanggil digitalWrite(PIN_LED, HIGH) sehingga LED menyala; jika '0' maka digitalWrite(PIN_LED, LOW) dan LED padam.
2. Serial.available() digunakan untuk memeriksa jumlah byte di buffer penerima sebelum membaca. Jika baris ini dihilangkan, Serial.read() dipanggil terus-menerus meskipun tidak ada data, mengembalikan nilai -1 yang diinterpretasikan sebagai karakter tidak dikenal, sehingga pesan error "Perintah tidak dikenal" akan muncul berulang kali tanpa henti dan mengacaukan Serial Monitor.
3. Untuk modifikasi LED berkedip saat menerima input '2': ditambahkan variabel boolean blinkMode dan variabel unsigned long lastBlinkTime. Ketika karakter '2' diterima, blinkMode diaktifkan. Selama blinkMode aktif, program memeriksa selisih millis() dengan lastBlinkTime setiap iterasi loop(); jika sudah melewati interval (misal 500 ms), status LED dibalik. Ketika '1' atau '0' diterima, blinkMode dinonaktifkan dan LED dikontrol sesuai perintah.Hasil Percobaan 2: Counter dengan Push Button

4. Sebaiknya menggunakan millis() daripada delay(). Fungsi delay() memblokir seluruh eksekusi program selama durasinya sehingga Arduino tidak dapat memproses input serial apapun selama LED berkedip, membuat sistem tidak responsif. Millis() mengembalikan waktu tanpa memblokir loop(), sehingga program dapat terus memeriksa input serial baru di setiap iterasi [3].

4.2 HASIL PERCOBAAN 3B: INTER INTEGRATED CIRCUIT (I2C)

Setelah program diupload dan rangkaian dirakit sesuai konfigurasi, LCD I2C berhasil menampilkan nilai ADC pada baris pertama dan representasi bar pada baris kedua. Saat potensiometer diputar ke kiri penuh, nilai ADC mendekati 0 dan bar tidak muncul. Saat diputar ke kanan penuh, nilai ADC mendekati 1023 dan bar memenuhi seluruh 16 posisi. Perubahan bar berlangsung mulus dan proporsional, membuktikan bahwa fungsi map() bekerja dengan benar dalam memetakan rentang ADC (0–1023) ke rentang bar (0–16).

Komunikasi I2C antara Arduino dan LCD berjalan melalui SDA (A4) untuk transfer data dan SCL (A5) untuk clock yang disinkronisasikan oleh Arduino sebagai master. Library LiquidCrystal_I2C menangani protokol komunikasi secara otomatis, termasuk pengiriman start condition, alamat slave (0x27), bit R/W, data perintah, dan stop condition [4][5].

Pertanyaan percobaan 3B:

1. Komunikasi I2C antara Arduino dan LCD: Arduino (master) mengirim sinyal START pada SDA saat SCL HIGH. Master mengirimkan 7-bit alamat (0x27) ditambah bit Write (0). LCD merespons dengan sinyal ACK. Setelah terbentuk koneksi, master mengirimkan byte perintah atau karakter. Modul backpack PCF8574 mengkonversi data I2C menjadi sinyal parallel 4-bit untuk kontroler LCD HD44780. Akhirnya master mengirimkan STOP untuk mengakhiri sesi [5].
2. Pin potensiometer harus dikonfigurasi tepat. Kaki kiri ke GND sebagai referensi tegangan bawah (0V), kaki kanan ke 5V sebagai referensi atas, dan kaki tengah ke A0 sebagai output pembagi tegangan. Jika kaki kiri dan kanan tertukar, arah putaran potensiometer akan terbalik: memutar ke kanan menurunkan ADC dan ke kiri menaikkannya, sehingga bar bergerak berlawanan dengan ekspektasi meski LCD tetap berfungsi.
3. Modifikasi menggabungkan UART dan I2C dilakukan dengan menambahkan Serial.print() pada setiap iterasi loop() untuk menampilkan nilai ADC, tegangan (nilai/1023.0 * 5.0 V), dan persentase (nilai/1023.0 * 100 %) ke Serial Monitor. LCD tetap menampilkan bar seperti sebelumnya. Kedua protokol berjalan bersamaan tanpa konflik karena beroperasi pada jalur hardware yang berbeda.
4. Tabel pengamatan nilai ADC pada Serial Monitor:

ADC	Volt(V)	Persen (%)
16	0	1%
21	0,10	2%
49	0,24	4%
74	0,36	7%
96	0,47	9%

4.3 JAWABAN PRAKTIKUM

1. Keuntungan UART: sederhana, hanya butuh 2 kabel (TX dan RX), tidak memerlukan clock eksternal, dan kompatibel hampir semua mikrokontroler. Kerugian UART: hanya mendukung komunikasi point-to-point, kecepatan terbatas, dan kedua perangkat harus dikonfigurasi baud rate yang sama. Keuntungan I2C: mendukung banyak perangkat pada dua kabel yang sama (multi-slave), hemat pin mikrokontroler, dan setiap slave memiliki alamat unik. Kerugian I2C: kecepatan lebih rendah dari SPI, memerlukan resistor pull-up, dan implementasi protokol lebih kompleks [4].
2. Alamat I2C pada LCD (misalnya 0x27 vs 0x20) berfungsi sebagai identitas unik perangkat di bus I2C. Ketika Arduino mengirimkan data ke LCD, ia terlebih dahulu mengirimkan alamat target ke bus. Hanya perangkat dengan alamat cocok yang merespons dengan ACK dan menerima data. Perbedaan alamat ditentukan oleh konfigurasi pin A0-A2 pada chip PCF8574 yang dapat diubah

dengan memindahkan jumper atau soldering pad di PCB modul [5].

3. Jika UART dan I2C digabung (input Serial Monitor, output ke LCD), alur kerjanya: pengguna mengetik di Serial Monitor, Arduino menerima data melalui UART (pin RX), memproses data yang diterima, kemudian mengirimkan output ke LCD menggunakan I2C (pin A4/A5). Arduino mengelola kedua protokol tanpa konflik karena keduanya beroperasi pada jalur hardware yang berbeda. UART menggunakan buffer berbasis interrupt sehingga data tersimpan otomatis, sementara I2C hanya aktif saat program memanggil fungsi lcd.print() [1][4].

5. KESIMPULAN

Praktikum Modul 3 tentang Protokol Komunikasi Serial telah berhasil dilaksanakan dalam dua percobaan. Percobaan 3A membuktikan bahwa komunikasi UART dapat digunakan untuk mengendalikan perangkat output secara real-time melalui Serial Monitor dengan perintah karakter. Mekanisme Serial.available() terbukti penting untuk menjaga stabilitas dan responsivitas sistem. Penggunaan millis() terbukti lebih tepat dibanding delay() untuk implementasi blink yang tidak memblokir komunikasi serial.

Percobaan 3B membuktikan bahwa protokol I2C memungkinkan komunikasi efisien antara Arduino dan LCD dengan hanya dua jalur (SDA dan SCL), menghemat pin secara signifikan. Nilai ADC dari potensiometer berhasil dipetakan dan ditampilkan sebagai bar yang proporsional, memvalidasi penggunaan analogRead() dan map() secara bersamaan. Secara keseluruhan, kedua protokol memiliki karakteristik berbeda: UART cocok untuk komunikasi debug dan point-to-point, sedangkan I2C lebih efisien untuk menghubungkan banyak perangkat periferal dalam satu sistem.

DAFTAR PUSTAKA

[1] Arduino LLC, "Serial Communication," *Arduino Reference Documentation*, Arduino LLC, 2024. [Online]. Tersedia di https://docs.arduino.cc/learn/communication/uart/, Diakses: 19 April 2026, pukul 10.00 WIB.

[2] SparkFun Electronics, "Serial Peripheral Interface (SPI)," *SparkFun Learn*, SparkFun Electronics, 2023. [Online]. Tersedia di https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi, Diakses: 19 April 2026, pukul 10.30 WIB.

[3] M. Margolis, B. Jepson, dan N. R. Weldin, *Arduino Cookbook: Recipes to Begin, Expand, and Enhance Your Projects*, 3rd ed., O'Reilly Media, Sebastopol, CA, USA, 2020.

[4] T. Leens, "An Introduction to I2C and SPI Protocols," *IEEE Instrumentation & Measurement Magazine*, vol. 12, no. 1, hal. 8–13, Feb. 2009.

[5] N. H. E. Weste dan D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson Education, Boston, MA, USA, 2011.